

CBO Choice Between Index & Full Scan: The Good, the Bad & the Ugly Parameters

Usually, the conclusion comes at the end. But here I will clearly show my goal: I wish I will never see the *optimizer_index_cost_adj* parameters again. Especially when going to 12c where Adaptive Join can be completely fooled because of it. Choosing between index access and full table scan is a key point when optimising a query and historically the CBO came with several ways to influence that choice. But on some systems, the workarounds have accumulated one on top of the other – biasing completely the CBO estimations. And we see nested loops on huge number of rows because of those wrong estimations.

Franck Pachot, Senior Consultant, dbi services

Full Table Scan vs Index Access

Full table scan is easy to cost. You know where the table is stored (the allocated segment up to the high water mark) so you just scan the segment blocks in order to find the information you are looking for. The effort does not depend on the volume of data that you want to retrieve, but only on the size of the table. Note that the size is the allocated size - you may have a lot of blocks to read even if the table is empty, just because you don't know that it is empty before you have reached the high water mark.

The good thing about Full Table Scan is that the time it takes is always the same. And because blocks are grouped in extents where they are stored contiguously, reading them from disk is efficient because we can read multiple blocks at a time. It's even better with direct-path and smart scan, or with in-memory option.

The bad thing is that reading all data is not optimal when you want to retrieve only a small part of information.

This is why we build indexes. You search the entry in the index and then go to the table, accessing only the blocks that may have relevant rows for your predicates. The good thing is that you do not depend on the size of your table, but only on the size of your result. The bad thing comes when you underestimate the

number of lookups you have to do to the table. Because in that case it may be much more efficient to full scan the whole table and avoid all those loops.

So the question is: do you prefer to read more information than required, but with very quick reads, or to read only what you need but with less efficient reads. People often ask for the threshold where an index access is less efficient than a full table scan. 15 years ago people were talking about 15% or 20%. Since then the 'rule of thumb' has decreased. Not because the behavior has changed, but I think it's just because the tables became bigger. Index access efficiency is not related to the table size, but only to the resulting rows. So those 'rules of thumb' are all wrong. In fact there are three cases:

- You need a few rows, and you accept that the time is proportional to the result, then go with index
- You need most of the rows, and you accept that the time is proportional to the whole data set, then full scan
- You're in between, then none are ok. Ideally, you need to change your data model to fit in one of the previous case.

But in the meantime, the optimizer has to find the least expensive access path.

Of course there are several variations where a Full Table Scan is not so bad even if you need only a small part of the rows (parallel query, Exadata SmartScan...). And there are other cases where index access is not that bad even to get lots of rows (covering index, well clustered index, prefetching/batching, cache, SSD...). But now let see how the optimizer is doing the choice.

Cost Based Optimizer

At the beginning, things were easy. If you can use an index, then use it. If you can't then full scan. Either you want to read everything, and you full scan (and join with sort merge join) or you want to retrieve only part of it and you access via index (and do a nested loop join). This sounds too simple, but it's amazing the number of application developers that are nostalgic of that RBO time. For small transactions it was fine. Remember, it was a time where there was no BI reporting tools, where you didn't have those 4 pages queries joining 20 tables, generated by those modern ORM, and tables were not so big. And if you had to optimize, denormalization was the way: break your data model for performance, in order to avoid joins.

Then came a very efficient join, the Hash Join which was very nice to join a big table with some lookup tables, even large ones. And at the same time came the Cost Base Optimizer. And people didn't understand why Oracle didn't support the brand new Hash Join with the old stable RBO. But the reason was just that it's impossible to do. How can you choose to join with Nested Loop index access or with Hash Join full table scan? There is no rule for that. It depends on the size. So you need Statistics. And you need the CBO.

Multiblock Read

Ok, you changed you optimizer mode to CBO. You were now able to do Hash Joins. You did not fear Full Table Scan anymore. What is the great power of full scans? You can read several blocks at once. The `db_file_multiblock_read_count` controls that number of blocks. And because the maximum I/O size at that time on most platforms was 64k, and default block is 8k, then the default value for `db_file_multiblock_read_count` was 8 blocks.

I'll illustrate the optimizer behaviour with a simple join between a 500 rows table and a 100000 rows table, forcing the join with hints in order to show how the Nested Loop Join and Hash Join cost is evaluated.

On my example, when we execute it, the nested loops is 3 times faster. Only when the first table reaches 1500 rows the nested loop response time is over the hash join. Nested Loop is the plan I want to be chosen by the optimizer for that query. Now, imagine I had that query 15 years ago. We will see how that query execution plan evolves with the versions of the CBO. So I set the optimizer to the 8i version and the `db_file_multiblock_read_count` to the value it had at that time: 8 blocks.

```
alter session set optimizer_features_enable='8.1.7';
alter session set db_file_multiblock_read_count=8;
```

And explain plan for both join methods:

Nested Loop in 8i with `db_file_multiblock_read_count=8`:

Id	Operation	Name	Rows	Bytes	Cost
0	SELECT STATEMENT		500	9000	1501
1	NESTED LOOPS		500	9000	1501
2	TABLE ACCESS FULL	A	500	4000	1
3	TABLE ACCESS BY INDEX ROWID	B	1	10	3
* 4	INDEX RANGE SCAN	I	1		2

Hash Join in 8i with `db_file_multiblock_read_count=8`:

Id	Operation	Name	Rows	Bytes	Cost
0	SELECT STATEMENT		500	9000	3751
* 1	HASH JOIN		500	9000	3751
2	TABLE ACCESS FULL	A	500	4000	1
3	TABLE ACCESS FULL	B	100K	976K	3749

Clearly the nested loop is estimated to be cheaper. This is the CBO default behavior up to 9.2.

How is the cost calculated? The cost estimates the number of I/O calls that has to be done.

Nested Loops has to do 500 index access and each of them has to read 2 index blocks and 1 table block. This is the cost=1500.

Hash Join has to Full Scan the whole table with 30000 blocks under the High Water Mark (we can see it in `USER_TABLES.BLOCKS`). Because we read 8 blocks at a time, the cost that estimates the number of I/O calls is $30000/8=3750$.

But then, at the time of 8i to 9i, the systems were able to do larger I/O. The maximum I/O size reached 1MB.

And in order to be able to do those large I/O we raised `db_file_multiblock_read_count` to 128 (when `db_block_size=8k`).

```
alter session set db_file_multiblock_read_count=128;
```

Let's see how the CBO estimates each join now.

Nested Loop in 8i with `db_file_multiblock_read_count=128`:

Id	Operation	Name	Rows	Bytes	Cost
0	SELECT STATEMENT		500	9000	1501
1	NESTED LOOPS		500	9000	1501
2	TABLE ACCESS FULL	A	500	4000	1
3	TABLE ACCESS BY INDEX ROWID	B	1	10	3
* 4	INDEX RANGE SCAN	I	1		2

Hash Join in 8i with `db_file_multiblock_read_count=128`:

Id	Operation	Name	Rows	Bytes	Cost
0	SELECT STATEMENT		500	9000	607
* 1	HASH JOIN		500	9000	607
2	TABLE ACCESS FULL	A	500	4000	1
3	TABLE ACCESS FULL	B	100K	976K	605

And now I have a problem. Hash Join looks cheaper. Cheaper in number of I/O calls, that's right. But it's not cheaper in time.

It's right that doing less I/O calls is better, because the latency is an important part of the disk service time. But we still have the same size to transfer. Reading 1MB in one I/O call is better than reading it in 16 smaller I/O calls. But we cannot cost those 1MB I/O as the same as one 8k I/O. This is the limit of costing the I/O calls. We now have to cost the time it takes. But that's for the next version (when 9i that introduced 'cpu costing').

This is what happened at that 8i time. We were able to do larger I/O but a lot of execution plans switched to Hash Join when it were not the right choice. We didn't want to lower *db_file_multiblock_read_count* and did not have a way to let the optimizer evaluate the cost as an estimate time.

So came a freaky parameter to influence the optimizer...

Cost Adjustment

The weird idea was: because Full Table Scan is under-estimated, let's under-estimate Index Access cost as well!

This is *optimizer_index_cost_adj* that defaults to 100 (no adjustment) but can change from 0 to 10000. Let's see what it does:

```
alter session set optimizer_index_cost_adj=20;
```

The Hash Join cost is the same as before (the under-evaluated cost=607) but now the Nested Loop is cheaper:

Id	Operation	Name	Rows	Bytes	Cost
0	SELECT STATEMENT		500	9000	301
1	NESTED LOOPS		500	9000	301
2	TABLE ACCESS FULL	A	500	4000	1
3	TABLE ACCESS BY INDEX ROWID	B	1	10	1
* 4	INDEX RANGE SCAN	I	1		1

The arithmetic is simple: we told the optimizer to under-evaluate index access to 20% of the calculated value. 300 instead of 1500. Nostalgic of RBO were happy. They had a mean to always favor indexes, even in CBO.

But this is a short-term satisfaction only, because now the cost is false in all the cases.

Why set the *optimizer_index_cost_adj* to 20%? It's an arbitrary way to lower the cost of index access as much as the cost of full table scan has been wrong. The goal is to compensate the ratio between multiblock read and single block read disk service times.

Of course, in hindsight, that was not a good approach. More and more decisions are based on the optimizer estimations and faking it with arbitrary value is not a good solution.

System Statistics

So the right approach is to change the signification of the cost. Estimating the number of I/O calls was fine when the size of I/O were all in the same ballpark. But now not all I/O are equal and we need to differentiate single block and multi block I/O. We need to estimate the time. The cost will now be the estimated time, even if for consistency with previous versions it will not be expressed in seconds but in number of equivalent single block reads that take the same time.

In addition to that, the optimizer tries to estimate also the time spend in CPU. This is why it is called 'cpu costing' even if the major difference is in the costing of multiblock I/O.

In order to do that, system statistics were introduced: we can calibrate the time it takes to do a single block I/O and a multiblock I/O. That was introduced in 9i but not widely used.

And calibration can also calculate a multiblock read count measured during a workload, or have the default value of 8 when *db_file_multiblock_read_count* if not explicitly set.

The idea is then not to set *db_file_multiblock_read_count*. The maximum I/O size will be used at execution time but the optimizer uses a more realistic value, either the default (which is 8) or the value measured during a workload statistics gathering. But what we often see in the real life is that the values that have been set once do remain for years even when not accurate anymore.

In 10g the 'cpu costing' became the default and uses default values if we didn't gather system statistics, based on a 10 millisecond seek time and a 4KB/millisecond transfer rate, and the default multiblock estimation is 8 blocks per I/O call.

So reading an 8KB block takes 10+2=12 milliseconds and reading 8 blocks take 10+16=26 milliseconds. This is how the choice between index access and table full scan can be evaluated efficiently.

```
alter session set optimizer_features_enable='10.2.0.5';
```

I've reset *optimizer_index_cost_adj* so that Nested Loop has the correct cost:

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		500	9000	1503 (1)	00:00:19
1	NESTED LOOPS		500	9000	1503 (1)	00:00:19
2	TABLE ACCESS FULL	A	500	4000	2 (0)	00:00:01
3	TABLE ACCESS BY INDEX ROWID	B	1	10	3 (0)	00:00:01
* 4	INDEX RANGE SCAN	I	1		2 (0)	00:00:01

You see the apparition of the estimated time.
Now cost is time. It is estimated to 1500 single block reads.

And Hash join now uses system statistics (I've reset `db_file_multiblock_read_count` as well):

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		500	9000	4460 (1)	00:00:54
* 1	HASH JOIN		500	9000	4460 (1)	00:00:54
2	TABLE ACCESS FULL	A	500	4000	2 (0)	00:00:01
3	TABLE ACCESS FULL	B	100K	976K	4457 (1)	00:00:54

And even if we do less I/O calls, Hash join is estimated to be longer. On multiblock reads, the transfer time is an important part of the response time, and this is what was not taken into account before system statistics.

So we have now the right configuration for the optimizer:

- `db_file_multiblock_read_count` not set
- `optimizer_index_cost_adj` not set
- accurate system statistics

This is the right configuration for all versions since 9i.

Unfortunately a lot of sites had moved to 'cpu costing' when upgrading to 10g but still keep some mystic value for `optimizer_index_cost_adj`. Thus they have a lot of inefficient reporting queries that are doing nested loops on large number of rows. This takes a lot of CPU and the response time increases as the volume increases. And people blame the instability of the optimizer, without realising that they explicitly give wrong input to the optimizer algorithm.

If this is your case, it's time to get rid of it. The problem it originally addressed empirically is now solved statistically. You should now check your system statistics and if SREADTIM and MREADTIM looks good (check `sys.aux_stats$`), then you should reset `optimizer_index_cost_adj` and `db_file_multiblock_read_count` to their default value.

If you didn't gather workload system statistics (which is the right choice if you're not sure that your workload is relevant) you won't see them in `sys.aux_stats$` but you can calculate from IOSEEKTIM and IOTFRSPEED:

- MBRC when not gathered is 8 when `db_file_multiblock_read_count` is not set (which is the right approach)
- SREADTIM when not gathered calculated as $\text{IOSEEKTIM} + \text{db_block_size} / \text{IOTFRSPEED}$
- And MREADTIM as $\text{IOSEEKTIM} + \text{db_block_size} * \text{MBRC} / \text{IOTFRSPEED}$

When you validate that those values are accurate, you can stop to fake the optimizer with arbitrary cost adjustments.

12c Adaptive Joins

We will be upgrading to 12c soon. And we will benefit of a very nice optimizer feature that will intelligently choose between Hash join and Nested Loop at execution time. This is a great improvement when the estimated cardinality is not accurate. The choice will be done at runtime, from the real cardinality.

But that decision is based on the cost. At parse time the optimizer evaluates the inflexion point where the cardinality is too high for a Nested Loop and when it is better to switch to a Hash Join. But if the cost for Nested Loop is under evaluated, then a Nested Loop will be used even for a high cardinality and that will be bad, consuming CPU to read always the same blocks.

Below is my adaptive execution plan on 12c.

Id	Operation	Name	Starts	E-Rows	Cost (%CPU)
0	SELECT STATEMENT		1		1503 (100)
* 1	HASH JOIN		1	500	1503 (1)
2	NESTED LOOPS		1		
3	NESTED LOOPS		1	500	1503 (1)
4	STATISTICS COLLECTOR		1		
5	TABLE ACCESS FULL	A	1	500	2 (0)
* 6	INDEX RANGE SCAN	I	500	1	2 (0)
7	TABLE ACCESS BY INDEX ROWID	B	500	1	3 (0)
8	TABLE ACCESS FULL	B	0	1	3 (0)

And from the optimizer trace (gathered with even 10053 or with `dbms_sqldiag.dump_trace`)

DP: Found point of inflection for NLJ vs. HJ: card = 1432.11

That means that the index access is the best approach as long as there is less than 1400 nested loops to do. If there is more, then Hash Join is better. The 'statistics collector' will count the rows at execution time to see if that inflexion point is reached.

But here is what happens if I keep my old cost adjustment inherited from the old 8i times:

```
alter session set optimizer_index_cost_adj=20;
```

The inflexion point is much higher:

```
DP: Found point of inflection for NLJ vs. HJ: card = 7156.65
```

If I have 5000 rows instead of 500 the execution will still do a Nested Loop, and that's obviously bad. That will do 15000 single block reads (2 index blocks and one table block per loop). Without a doubt it is far more efficient to read the 30000 blocks from table with a full table scan doing large I/O calls.

Conclusion

Using *optimizer_features_enable* like a time machine we were able to see how the optimizer has evaluated the cost of index vs. full scan in the past. But there is an issue that is current. A lot of databases still have old settings, and a lot of software editors still recommend those old settings. They finally gave up with RBO because they cannot recommend a desupported feature. But probably because of the fear of change, they still recommend this old cost adjustment setting.

However the only reason for it it disappeared with system statistics, years ago. So it's time to stop faking the CBO. Today the CBO can do really good choices when having good input. Since 10g, the good is System Statistics, the bad is RBO, and the ugly is *optimizer_index_cost_adj*. You are in 10g, 11g or even 12c, then choose the good and don't mix it with an ugly setting inherited from the past. ■



ABOUT THE AUTHOR

Franck Pachot

Senior Consultant, dbi services

Franck Pachot is senior consultant at dbi services in Switzerland. He has 20 years of experience in Oracle databases, all areas from development, data modeling, performance, administration, training. He tries to leverage knowledge sharing in forums, publications, presentations, and became recently an Oracle Certified Master.

Autumn Special Interest Group Meetings

September

- 17th** UKOUG Business Intelligence & Reporting Tools SIG, London
- 17th** UKOUG Solaris SIG Meeting, London
- 18th** UKOUG Public Sector HCM Customer Forum, Solihull
- 24th** OUG Ireland BI & EPM SIG Meeting, Dublin
- 24th** OUG Ireland HCM SIG Meeting, Dublin
- 24th** OUG Ireland Technology SIG Meeting, Dublin
- 24th** UKOUG RAC Cloud Infrastructure & Availability SIG, Solihull
- 25th** UKOUG Oracle Projects SIG, London

October

- 7th** UKOUG Public Sector Applications SIG Meeting, Solihull
- 9th** UKOUG Application Server & Middleware SIG, Reading
- 14th** UKOUG Taleo SIG Meeting, London
- 15th** UKOUG Solaris SIG Meeting, London
- 21st** UKOUG Database Server SIG, Reading
- 22nd** UKOUG Supply Chain & Manufacturing SIG, Solihull
- 23rd** UKOUG HCM SIG, Solihull
- 23rd** UKOUG Partner Forum, London
- 23rd** UKOUG Partner of the Year Awards 2014, London

November

- 4th** UKOUG Public Sector Financials Customer Forum, London
- 6th** UKOUG Apps DBA for OEBS, London
- 12-13th** UKOUG JD Edwards Conference & Exhibition 2014, Oxford
- 18th** UKOUG Application Express (APEX) SIG Meeting, London
- 19th** UKOUG Solaris SIG Meeting, London
- 27th** UKOUG Public Sector HCM Customer Forum - Workshop, Solihull

December

- 8-10th** UKOUG Applications Conference & Exhibition 2014, Liverpool
- 8-10th** UKOUG Technology Conference & Exhibition 2014, Liverpool
- 9th** UKOUG Primavera 2014, Liverpool
- 17th** UKOUG Solaris SIG Meeting, London